

LABORATORIO DE INGENIERÍA CON IA

Guía Maestra de Spec-Driven Development, Arnesees, Agentes, Guardrails, Skills y MCP

Manual integral de arquitectura moderna y gobernanza algorítmica: desde la especificación viva como única fuente de verdad (SSOT) hasta el arnés sensorial de evaluación, los guardrails automatizados en 5 fases, el enjambre multi-agente, los skills especializados y el protocolo universal MCP.

AUTOR / SPONSOR & DESARROLLADOR LÍDER:

Christian Vargas A.

PROYECTO DE REFERENCIA:

SuperCarrito SDD (React 18 + Vite + Tailwind CSS v4)

COPILOTO TÉCNICO AI:

Antigravity (Google DeepMind)

FECHA Y VERSIÓN:

Octubre 2026 | Versión 4.0 Consolidada (47 Guardrails, 9 Tests E2E, 22 ADRs, Catálogo de 7 Markdowns y 21 Q&A)

1. ¿Qué es Spec-Driven Development (SDD)?

El **Desarrollo Guiado por Especificaciones (SDD)** es una metodología de ingeniería de software en la cual **una especificación viva, formal y versionada actúa como la Única Fuente de Verdad (Single Source of Truth – SSOT)** antes de escribir o modificar una sola línea de código de producción.

La Regla de Oro del SDD

"Si no está en la Especificación (`docs/SPEC.md`), no existe. Si está en la Especificación, el software está obligado a cumplirlo matemática y sensorialmente."

La Metáfora de la Construcción Civil

Desarrollo Tradicional / Caótico con IA

Pedirle a un albañil que levante paredes en un terreno vacío diciéndole: *"Hazme una sala bonita y luego vemos dónde va el baño"*. El resultado inevitable son demoliciones continuas, vigas mal calculadas y colapso estructural.

Spec-Driven Development (SDD)

El arquitecto diseña el plano formal (`SPEC.md`), calcula cargas de ingeniería (`ARCHITECTURE.md`) y define tolerancias de materiales (`RULES.md`). Ningún operario pega un solo bloque sin verificar el plano.

2. El Arnés de Pruebas (The Test Harness) – El Órgano Sensorial

Un Modelo de Lenguaje (LLM) es un procesador de texto probabilístico: no tiene ojos, manos ni navegador para comprobar físicamente si un botón hace clic o si una tabla se actualizó. El **Arnés de Pruebas (Test Harness)** es el andamiaje instrumental que le otorga **percepción sensorial y retroalimentación empírica en tiempo real**.

```
| ARQUITECTURA DEL ARNÉS SDD | | | |
[ docs/SPEC.md ] → Define Contratos y Requerimientos (RF / RNF) | | | | ▼ | | [ tests/harness.html ]
← | | [ tests/e2e.test.jsx ] → Inyecta Estímulo Sensorial (Click/Dom) | | | | ▼ | | [ Código
de la App ] → | | | | ▼ | | Aserción Matemática: ¿Esperado = Obtenido? | | — ● VERDE (PASS):
Contrato Validado y Certificado | | — ● ROJO (FAIL): Discrepancia detectada → Feedback al Agente |
```

La Metáfora de la Cocina Gourmet

La Receta Maestra (**SPEC.md**)

Es la **Ficha Técnica Oficial**. Define ingredientes al gramo, orden exacto de cocción y temperaturas.

El Chef Asistente (La IA / Agente)

Ejecuta la receta siguiendo instrucciones. Es talentoso pero puede distraerse o alucinar ingredientes.

El Arnés de Pruebas (**harness.html** / Vitest)

Es el **Inspector de Calidad**. Termómetro digital, refractómetro y báscula: nada sale a la mesa sin aprobación.

La Bitácora de Memoria (**MEMORY.md**)

Es el **Cuaderno del Chef**. Registra incidentes, lecciones aprendidas y mejoras de recetas (ADRs).

🔗 3. El Ciclo de Vida: Rojo → Verde → Refactor

FASE	COLOR	ACCIÓN EN EL LABORATORIO	PROPÓSITO / OBJETIVO
1. Fase Roja	● FAIL	Diseñar la prueba antes de programar y verla fallar con un error explícito.	Demostrar que la prueba es real y detecta la ausencia de la función.
2. Fase Verde	● PASS	Escribir el código mínimo necesario para que la prueba pase a verde.	Alcanzar el cumplimiento estricto del contrato sin sobre-ingeniería.
3. Refactorización	● CLEAN	Reorganizar taxonomía, limpiar deuda técnica, optimizar código.	Mejorar la calidad sin miedo: el arnés garantiza que nada se rompió.

⚠️ La Trampa de las Pruebas Falsas Positivas

Si creas una prueba y pasa en verde de inmediato sin que hayas programado nada, **la prueba no sirve**. La Fase Roja es la garantía científica de que la prueba realmente detecta la falta de la función.

[Requerimiento en SPEC.md] → [Caso de Prueba en Arnés] | ▼ ● FASE ROJA: FALLO (Comprobación Científica) | ▼ [Programar Solución] | ▼ ● FASE VERDE: ÉXITO (Cumplimiento del Contrato) | ▼ ● REFACTORIZAR & DoD (Guardrails 47/47 + ADR en MEMORY.md)

🚧 4. Guardrails: Las Barandillas de Seguridad (Explicado para Todos)

La palabra **Guardrail** significa literalmente la **barandilla de contención** metálica que ves al borde de una carretera en la montaña. Tú sabes manejar, pero si hay hielo, la barandilla **físicamente te impide caer al abismo**.

4 Ejemplos Visuales de la Vida Real:

1. La puerta del microondas

Si abres la puerta mientras calienta, el microondas **se apaga en el acto**. Es un guardrail físico: no confía en que recuerdes apagarlo; bloquea el peligro por diseño.

2. Las rueditas de la bicicleta infantil

No pedalean por el niño, pero si la bicicleta se inclina peligrosamente hacia un costado, tocan el suelo e impiden una caída grave.

3. El enchufe de 3 clavijas (con polo a tierra)

Tiene una clavija redonda más larga. Es geométricamente imposible enchufarlo al revés; su propia forma actúa como guardrail.

4. El script automatizado (check:guardrails)

Escanea **47 verificaciones en 5 fases en ~50ms** e impide que se suba código que viole el idioma español, toque storage directo o rompa aserciones.

Los 4 Niveles de Guardrails en Desarrollo con IA:

- 1. **Guardrails de Instrucción** (`app/AGENTS.md` , `docs/RULES.md`): Reglas escritas que prohíben alucinar o inventar APIs.
- 2. **Guardrails Estructurales** (Scripts y CI/CD – `scripts/guardrails.js`): 47 verificaciones automáticas por terminal antes de compilar o hacer commit.
- 3. **Guardrails de Acero** (Git Pre-commit Hook): Bloqueo físico en el sistema de control de versiones: si un solo guardrail falla, el commit es abortado de raíz.

- **4. Guardrails de UX (ConfirmModal):** Modales táctiles flotantes que impiden desmarcar compras por error al caminar ("anti-dedazos").

5. Skills (Habilidades): La Carpeta de Especialidad de la IA

Un **Skill** es un paquete modular de instrucciones avanzadas, scripts auxiliares y plantillas que enseñan a la IA a realizar una tarea especializada sin necesidad de reentrenar el modelo base.

Carpeta de Recetas del Chef

El modelo base sabe cocinar en general; al activar el skill `generative_ui` o `automation`, abre el manual técnico del chef pastelero y opera con precisión de especialista.

6. MCP (Model Context Protocol): El USB-C Universal de la IA

El **Model Context Protocol (MCP)** es un estándar abierto desarrollado por Anthropic para permitir que los asistentes de IA se conecten de forma segura a fuentes de datos locales, servidores, repositorios GitHub, bases de datos SQL y herramientas empresariales mediante un protocolo cliente-servidor estandarizado.

7. Arquitectura de Agentes, Subagentes y Slash Commands

- **Humano (Christian Vargas A.):** Conduce el proyecto, define requerimientos de negocio y aprueba las especificaciones.
- **Agente Orquestador (Antigravity):** Mantiene la visión general, conversa con el humano, actualiza `MEMORY.md` y supervisa el cumplimiento del contrato.
- **Subagentes Especializados:** Asistentes efímeros que el orquestador despierta para tareas aisladas (Investigador para leer documentación sin saturar la conversación, Tester para certificar calidad).
- **Slash Commands (/):** Atajos rápidos en la caja de texto (como `/plan`, `/goal`, `/boost`) para activar flujos de trabajo de un solo toque.

 8. Matriz Comparativa y Diccionario Conceptual Rápido

CONCEPTO	ANALOGÍA DE LA VIDA REAL	FUNCIÓN EN EL PROYECTO SUPERCARRITO
Spec-Driven Development (SDD)	El plano del arquitecto antes de colocar ladrillos.	<code>docs/SPEC.md</code> como Única Fuente de Verdad (SSOT v2.16.0).
Arnés de Pruebas (Harness)	El banco de pruebas de choque o el inspector de cocina.	<code>tests/harness.html</code> para validación sensorial matemática.
Guardrails Duales (CLI + Web)	La barandilla de autopista y el tablero de control de mando.	<code>scripts/guardrails.js</code> (terminal) y <code>tests/guardrails.html</code> (web) con 47 verificaciones en 5 fases.
Guardrail de Acero (Git)	La barrera de control que bloquea el paso sin ticket válido.	<code>.githubhooks/pre-commit</code> impidiendo físicamente commits si una regla falla.
Arnés Sensorial E2E	El dedo real del usuario interactuando con la pantalla.	<code>tests/e2e.test.jsx</code> con Vitest y JSDOM simulando 9 flujos de usuario completos.
Patrón del Puntero Maestro	El índice general de una biblioteca clasificada.	<code>app/AGENTS.md</code> guiando hacia la documentación en <code>docs/</code> en orden estricto.
Dashboard 360° (Power BI)	La torre de control de un aeropuerto internacional.	<code>AuditoriaHub.jsx</code> opción 1: métricas visuales ejecutivas en 5 tarjetas KPIs.
Laboratorio de Caos	El simulacro de terremoto con bomberos e ingenieros.	Inyector de mutaciones deliberadas (corrupción JSON, violaciones a storage) para probar auto-sanación.
Contrastes WCAG AAA	La señalización vial reflectiva de alta visibilidad.	Normalización de tarjetas claras (<code>#faf5ff</code> , <code>#f0f9ff</code>) y consolas oscuras con texto nítido.
Skills	La carpeta de recetas de alta pastelería del chef.	Paquetes de instrucciones avanzadas para tareas expertas.
MCP	El conector universal USB-C.	Protocolo estándar para conectar la IA a bases de datos y herramientas sin plugins propietarios.
ADRs (Architecture Decisions)	El libro de actas notariadas de una junta directiva.	Registros numerados en <code>docs/MEMORY.md</code> (ADR-01 a ADR-22).

9. Caso de Estudio Real: SuperCarrito SDD (Arquitectura Módulo a Módulo)

Módulo 1: Portada y Navegación Minimalista (HomeHub.jsx – RF-5.1 / RF-5.2)

Puerta de entrada táctil estructurada en los 3 momentos del usuario: Paso 1 (Preparar despensa y listas en casa), Paso 2 (¡Vamos al Súper! en tienda) y Paso 3 (Laboratorio SDD y Auditoría). Header limpio que preserva exclusivamente identidad, selector Claro/Oscuro y retorno al inicio.

Módulo 2: Centro Unificado de Gestión (GestionHub.jsx – RF-5.4 / RF-5.5 / RF-3.1)

Estructura ergonómica estilo AdminLTE con dos columnas: barra lateral para alternar entre Catálogo de Despensa y Mantenimiento de Listas con contadores en tiempo real. Incorpora selector con casillas de verificación (checkboxes) y botón de carga en lote simultáneo (RF-3.1).

Módulo 3: Modo Compra Táctil con Confirmación Anti-dedazos (ActiveShopping.jsx – RF-4.6)

Diseño ultra-lineal de una sola columna optimizado para usarse con una mano caminando por los pasillos. Erradicación del `window.confirm` nativo a favor de un modal táctil flotante (`ConfirmModal.jsx`) con botones gigantes amigables para el pulgar.

Módulo 4: Centro Integrado de Auditoría (AuditoriaHub.jsx – RF-5.3 a RF-5.10)

Panel lateral AdminLTE con 7 consolas especializadas: 1. Dashboard 360° Ejecutivo estilo Power BI, 2. Guardrails Maestros (47/47 en vivo), 3. Arnés Sensorial interactivo, 4. Guardrail de Acero en Git (Pre-commit hook certificado), 5. Suite Sensorial E2E con 9 flujos Vitest, 6. Enjambre Multi-Agente y 7. Laboratorio de Caos e Inmunidad SDD.

Módulo 5: Contrastes Adaptativos y Tokens WCAG AAA (RF-5.2 / ADR-22)

Normalización visual de contenedores claros de alta legibilidad (`#faf5ff` , `#f0f9ff` , `#f0fdf4`) para tarjetas ejecutivas y consolas oscuras obsidian (`#0b0f19`) para logs técnicos, erradicando fondos oscuros fuera de lugar.

Módulo 6: Patrón del Puntero Maestro (app/AGENTS.md – RF-5.7)

Limpieza absoluta de la raíz del proyecto. Solo reside `app/AGENTS.md` como Constitución Operativa y Puntero Maestro, centralizando toda la documentación y estándares en el directorio `docs/` con tabla de lectura obligatoria.

Diagrama de Capas del Sistema (C4 Component Diagram)

Tailwind v4) | App.jsx (Orquestador de Vistas) | | HomeHub.jsx (Portada con flujo cronológico en 3 pasos) | | GestionHub.jsx (AdminLTE: Catálogo y Listas con Checkboxes) | | ActiveShopping.jsx (Modo Súper Táctil de 1 columna) | | ConfirmModal (Guardrail de UX Anti-dedazos - RF-4.6) | | AuditoriaHub.jsx (Dashboard 360°, 47 Guardrails, Caos, E2E) |

Toggle) ▾ | Invocaciones desacopladas (CRUD /

SERVICIO | SupermarketStorage (src/services/storage.js) | | - Reglas de negocio e integridad referencial en cascada | | - Tolerancia a JSON corrupto (RNF-02) | | - Migración silenciosa al vuelo v2.0 → v2.1 (RNF-03) | | Persistencia aislada ▾ |

localStorage ('supermarket_app_db_v2') [Navegador] | | MockStorage (Sandbox en memoria para Guardrails y Vitest) |

Diagrama del Guardrail de Acero (Git Pre-commit Hook)

```
[ Desarrollador / Copiloto IA ejecuta "git commit" ] | ▼ [ .github/hooks/pre-commit intercepta la acción físicamente ] | ▼ [ node scripts/guardrails.js (47 Verificaciones en <50ms) ] |  
└───┘ ▼ ▼ ¿Todo 100% Verde? ¿Algún fallo o regla rota? | | ▼ ▼  Commit AUTORIZADO  
 Git ABORTA la operación (Código certificado) (Imposible subir código roto)
```



10. Checklist Profesional y Protocolo de Trabajo Definitivo

1. **Definir y Acordar:** Entender el problema de negocio con el usuario y sponsor.
2. **Documentar en la Spec:** Redactar la regla en `docs/SPEC.md` (subir versión SemVer).
3. **Diseñar el Caso de Prueba:** Agregar la aserción en `tests/harness.html` o `tests/e2e.test.jsx` con código `RF-*`.
4. **Comprobar el Fallo Rojo:** Re-ejecutar el arnés y certificar el fallo científico (Fase Roja).
5. **Escribir el Código Mínimo:** Modificar los componentes y servicios en `src/` hasta lograr Verde.
6. **Ejecutar Guardrails:** Correr `npm run check:guardrails` en terminal (47/47 en verde) y suite sensorial E2E (9/9 flujos).
7. **Registrar en Memoria:** Guardar el ADR y lecciones aprendidas en `docs/MEMORY.md`.
8. **Commit Semántico en Git:** El Pre-commit Hook valida los 47 guardrails y autoriza el commit atómico.

📁 11. Catálogo Maestro de Archivos Markdown (*.md) del Proyecto

En una arquitectura gobernada por **Spec-Driven Development (SDD)**, los archivos Markdown (*.md) no son simples notas pasivas: son **artefactos ejecutables de gobernanza, contratos de verdad y compuertas de seguridad** que dictan el comportamiento estricto tanto del equipo humano como de los agentes de Inteligencia Artificial.

📖 Tabla Sinóptica de los 7 Artefactos Markdown Oficiales

ORDEN	ARCHIVO	UBICACIÓN	ROL ARQUITECTÓNICO	AUDIENCIA	CRITICIDAD
0°	app/AGENTS.md	app/AGENTS.md	Constitución Operativa y Puntero Maestro	Agentes de IA	🔴 Innegociable
1°	app/docs/SPEC.md	app/docs/SPEC.md	Única Fuente de Verdad (SSOT v2.16.0)	Sponsor, IA y QA	🔴 Suprema
2°	app/docs/ARCHITECTURE.md	app/docs/ARCHITECTURE.md	Arquitectura de Software y Diagramas C4	Arquitectos e IA	🟡 Alta
3°	app/docs/RULES.md	app/docs/RULES.md	Estándares de Código y Definition of Done	Desarrolladores e IA	🔴 Innegociable
4°	app/docs/MEMORY.md	app/docs/MEMORY.md	Bitácora de Memoria, ADRs 01-22 y Roadmap	IA y Equipo	🟡 Alta
Infra	app/README.md	app/README.md	Portada Técnica y Guía de Despliegue	Desarrolladores	🟡 Media
Docs	GUIA_MAESTRA.md	info-docs/	Guía Maestra Pedagógica y Conceptual	Estudiantes y Líderes	🟢 Formativa

📊 Diagrama de Jerarquía Documental del Repositorio

```
[ Nivel 0: Entrada y Constitución ] | ▼ app/AGENTS.md (Puntero Maestro y Constitución Operativa) |
└───────────────────────────────────┴───▶ [ Nivel 1: Base de Conocimiento Centralizada ] app/README.md
(Despliegue) | ── 1° docs/SPEC.md (SSOT - Contratos de Negocio RF/RNF) ── 2° docs/ARCHITECTURE.md
(Diagramas C4 y Flujos) ── 3° docs/RULES.md (Estándares y DoD) ── 4° docs/MEMORY.md (ADRs 01-22 y
Roadmap Activo) | ▼ [ Nivel 2: Formación y Pedagogía ] info-docs/GUIA_MAESTRA_SDD_AGENTES_ARNESES.md
(Manual Teórico + 21 Q&A)
```

📄 Desglose Exhaustivo de los 7 Artefactos:

1. app/AGENTS.md (Constitución Operativa y Puntero Maestro)

Es el artefacto de **orden 0º**. Intercepta la atención del agente al iniciar, prohibiendo tocar código sin antes consultar la especificación. Centraliza los comandos rápidos de verificación (`npm run check:guardrails` y `npm run test:e2e`) y los estándares de ingeniería.

2. app/README.md (Ficha Técnica y Despliegue)

Carta de bienvenida para desarrolladores y evaluadores humanos. Resume el stack técnico (React 18, Vite, Tailwind v4, Vitest), el árbol de directorios de `app/` y las instrucciones de terminal para levantar el servidor y las pruebas.

3. app/docs/SPEC.md (Especificación Principal – SSOT)

El **artefacto supremo del sistema (Orden 1º)**. Contiene la versión SemVer formal (v2.16.0), el changelog histórico, el modelo de datos relacional en español y los contratos funcionales (RF-1.1 a RF-5.10) y no funcionales (RNF-01 a RNF-04).

4. app/docs/ARCHITECTURE.md (Arquitectura y Diagramas C4)

Artefacto de **orden 2º**. Modela mediante C1, C2, C3 y C4 cómo interactúan las piezas de software, la topología del enjambre multi-agente, el algoritmo de auto-sanación ante JSON corrupto y el embudo de seguridad del panel ejecutivo.

5. app/docs/RULES.md (Estándares y Definition of Done)

Artefacto de **orden 3º**. El código penal del proyecto: impone español obligatorio para entidades de negocio, prohíbe el uso de `localStorage` en componentes de UI y dicta la Definición de Terminado innegociable.

6. app/docs/MEMORY.md (Bitácora de Decisiones, ADRs y Roadmap)

Artefacto de **orden 4º**. Es la memoria a largo plazo: conserva los **22 ADRs formales (ADR-01 a ADR-22)**, 21 lecciones aprendidas de ingeniería y el roadmap activo con sus 32 pasos completados.

7. info-docs/GUIA_MAESTRA_SDD_AGENTES_ARNESES.md (Guía Pedagógica y Conceptual)

El libro de texto integral y manual formativo del proyecto. Traduce conceptos de alta ingeniería a metáforas cotidianas, analiza el caso SuperCarrito en detalle y provee el banco extendido de 21 preguntas y respuestas.

💡 12. Preguntas y Respuestas Frecuentes (Banco Extendido de Repaso – 21 Consultas Clave)

Compilación exhaustiva de las consultas reales planteadas por Chris durante nuestras sesiones de laboratorio, con las explicaciones conceptuales y prácticas que sustentan la arquitectura.

P1 ¿Por qué no programamos directamente la interfaz o la base de datos en vez de escribir primero la especificación?

Porque sin especificación previa, la IA programa por adivinanza estadística (alucinación funcional). Cuando programas primero la UI, descubres a mitad de camino que faltan campos relacionales o que el borrado en cascada rompe la vista. En SDD, la especificación en `docs/SPEC.md` es el plano formal aprobado: programar se convierte en rellenar contratos exactos, reduciendo retrabajos a cero.

P2 ¿Qué diferencia hay entre una prueba unitaria común y un "Arnés de Pruebas"?

Una prueba unitaria típica verifica una función aislada fuera de contexto. El **Arnés de Pruebas (Harness)** es un órgano sensorial completo: levanta la aplicación, simula el entorno físico de ejecución, inyecta estímulos secuenciales de usuario (flujos de interacción) y ofrece un tablero visual con telemetría en tiempo real accesible tanto para el copiloto de IA como para el auditor humano.

P3 ¿Por qué es tan peligroso saltarse la "Fase Roja" en el ciclo de pruebas?

Saltarse la fase roja es como instalar una alarma contra incendios y nunca probar si suena cuando hay humo. Si la prueba pasa en verde sin haber programado la función, estás ante un **falso positivo**: una prueba mal escrita que siempre aprueba, dando una falsa sensación de seguridad que oculta bugs críticos en producción.

P4 ¿Qué sucede si la IA inventa una función o atributo que no estaba en SPEC.md?

Ocurre una violación de contrato. Los Guardrails automáticos (Fase 4: Linter de Dominio) y el arnés detectan la discrepancia y el sistema falla. En SDD, cualquier adición técnica debe nacer como una propuesta en `docs/SPEC.md`, aprobada por el sponsor e incrementando la versión SemVer formal.

P5 ¿Por qué los Guardrails son más rápidos que las pruebas tradicionales en navegador?

Porque los **47 Guardrails Maestros** se ejecutan puramente en Node.js analizando el árbol sintáctico (AST) de los archivos, expresiones regulares de texto y pruebas en memoria (sandbox MockStorage), tardando **menos de 50 milisegundos**, sin la sobrecarga de levantar ventanas gráficas de navegador.

P6 ¿Qué es un Skill en Antigravity y cuándo conviene activarlo?

Un Skill es una carpeta modular de instrucciones y utilidades que capacita a la IA en una especialidad concreta (por ejemplo, `generative_ui` para crear widgets visuales o `automation` para tareas programadas). Conviene activarlo cuando la tarea requiere conocimiento experto que excede el modelo base.

P7 ¿En qué se diferencia MCP (Model Context Protocol) de una API tradicional?

Una API REST o GraphQL tradicional requiere código cliente personalizado para cada endpoint y proveedor. **MCP** es un conector estándar universal (como el USB-C): un solo protocolo mediante el cual la IA descubre herramientas, recursos y esquemas de datos expuestos por cualquier servidor compatible sin adaptadores ad-hoc.

P8 ¿Por qué utilizamos subagentes en lugar de hacer todo con el agente principal?

Para evitar la saturación de la ventana de contexto. Cuando un subagente investiga código o lee logs masivos, lo hace en un hilo aislado; solo devuelve el resumen sintetizado al agente orquestador, manteniendo la conversación principal limpia, rápida y enfocada.

P9 ¿Qué rol cumplen los Slash Commands en el flujo diario de trabajo?

Son aceleradores cognitivos: comandos como `/plan`, `/goal` o `/boost` activan directamente flujos de trabajo multi-paso prediseñados, evitando redactar instrucciones repetitivas en cada solicitud.

P10 ¿Por qué prohibimos a los componentes de React llamar directamente a localStorage?

Por el principio de **Aislamiento de Capas** (Regla innegociable verificada por la Fase 4 de Guardrails). Si la UI llama directo a `localStorage`, se vuelve imposible migrar de motor, agregar validaciones o probar en memoria. Al pasar por `SupermarketStorage`, centralizamos integridad referencial, sanitización y tolerancia a fallos.

P11 ¿Qué es un ADR (Architectural Decision Record) y por qué los registramos en MEMORY.md?

Un ADR es un registro formal que documenta el contexto, la decisión técnica tomada y sus consecuencias. En nuestro laboratorio contamos con **22 ADRs formales (ADR-01 a ADR-22)** que aseguran que el razonamiento detrás de decisiones críticas (como adoptar Tailwind v4, modal anti-dedazos o tokens WCAG AAA) no se olvide entre sesiones.

P12 ¿Cómo garantiza el sistema que ninguna tarea se dé por terminada antes de tiempo?

A través de la **Definición de Terminado (DoD)** en `docs/RULES.md`. Una tarea solo está lista si tiene: 1. Especificación formal, 2. Caso en arnés, 3. Fase roja certificada, 4. Fase verde alcanzada, 5. Los 47 guardrails aprobados, 6. 9 flujos E2E aprobados, 7. ADR asentado en `docs/MEMORY.md` y 8. Commit validado por el Pre-commit Hook.

P13 ¿Por qué eliminamos window.confirm en el modo de compra táctil en tienda?

Porque el cuadro de diálogo nativo del navegador (`window.confirm`) bloquea el hilo de ejecución, es inaccesible visualmente y tiene botones pequeños propensos a pulsaciones accidentales con una sola mano. Fue reemplazado por `ConfirmModal.jsx` (RF-4.6), un modal flotante con botones táctiles gigantes optimizados para el pulgar.

P14 ¿Cómo opera la auto-recuperación ante datos corruptos en el almacenamiento (RNF-02)?

El servicio `SupermarketStorage` envuelve todas las lecturas en bloques `try/catch` con validación de esquemas. Si un usuario o script introduce JSON malformado, el servicio detecta la anomalía, restaura silenciosamente las tablas base vacías e impide que la aplicación caiga en pantalla blanca.

P15 ¿Cómo funciona la migración transparente de esquemas al vuelo (RNF-03)?

Al arrancar la aplicación, el servicio detecta si existen tablas en inglés de la v2.0 (`products` , `lists` , `list_items`). Si las encuentra, ejecuta un mapper en memoria que traduce claves y estados al español canónico (`productos` , `listas` , `enCarrito`), persistiendo el nuevo formato sin pérdida de datos para el usuario.

P16 ¿Por qué adoptamos la navegación lateral estilo AdminLTE en el módulo de gestión (RF-5.5)?

Para separar limpiamente las responsabilidades visuales. El Catálogo de Despensa y las Listas de Compras tienen naturalezas distintas: la barra lateral permite saltar entre ambas con contadores numéricos en vivo sin recargar la página ni saturar la pantalla con formularios apilados.

P17 ¿Qué ventaja tiene contar con un panel de Guardrails dentro de la propia interfaz web (RF-5.3)?

Democratiza la auditoría técnica. Cualquier usuario, evaluador o sponsor puede verificar visualmente la salud del sistema (los 47 puntos en verde) con un solo clic en la aplicación web, sin necesidad de abrir una terminal o instalar Node.js.

P18 ¿Cómo se configuran los ganchos de Git mediante `.githubhooks` y `core.hooksPath` para trabajar en equipo?

La carpeta `.git/hooks/` no se versiona en Git. Al crear la carpeta versionada `.githubhooks/` con el script ejecutable `pre-commit` y ejecutar `git config core.hooksPath .githubhooks` (automatizado con `npm run setup:hooks`), el "Guardrail de Acero" se comparte con todo el equipo, forzando la evaluación de los 47 guardrails en cualquier máquina que clone el proyecto.

P19 ¿Cómo logra Vitest + JSDOM ejecutar las pruebas E2E en ~2 segundos sin navegadores pesados?

En lugar de levantar navegadores Chromium reales (como Cypress o Selenium) que tardan decenas de segundos, **JSDOM** emula toda la API del DOM puramente en memoria dentro de Node.js. Al combinarse con React Testing Library, simula eventos táctiles, clics y cambios de vista con fidelidad absoluta a la velocidad de la CPU en milisegundos (9 flujos completos en ~2.5s).

P20 ¿Por qué organizamos la aplicación en un flujo cronológico de 3 momentos (En casa, En tienda, Laboratorio)?

Porque el software debe alinearse con la psicología humana y no con la estructura de sus tablas relacionales. El usuario vive 3 momentos claros: 1. Planificar la compra en casa (Paso 1: Gestión), 2. Hacer las compras caminando en tienda (Paso 2: Modo Súper Táctil) y 3. Auditar la calidad del sistema (Paso 3: Laboratorio SDD). Esta arquitectura mental elimina fricción y maximiza la usabilidad.

P21 ¿Cuándo usar una Especificación Centralizada (docs/SPEC.md) versus Specs Locales por Componente, y quién las redacta?

1. **¿Quién redacta?:** El **Copiloto AI (Antigravity)**, guiado por la visión y directivas de negocio de Christian Vargas A. Tú no tienes que redactar Markdown manualmente: Antigravity actualiza `docs/SPEC.md`, asienta el ADR en `docs/MEMORY.md` y construye el código.
2. **¿Centralizado o Local?:** Para el 90% de las aplicaciones completas (como SuperCarrito), el **patrón centralizado en `docs/SPEC.md` es el estándar óptimo**: evita la fragmentación de contexto del LLM y ofrece una visión integral del modelo relacional. Los specs locales co-localizados se reservan para design systems aislados o monorrepos masivos.

SuperCarrito SDD – Laboratorio de Aprendizaje y Excelencia en Ingeniería

Documento maestro generado para Christian Vargas A. en colaboración con Antigravity (Google DeepMind). Versión 4.0 Consolidada.